Davante

14

PROGRAMACIÓN

# Gestión de ficheros binarios y estructuras jerárquicas anidadas

# ÍNDICE

/ 1. Introducción y contextualización práctica 4

/ 2. Lectura de ficheros binarios 5

/ 3. Escritura de ficheros binarios 6

/ 4. Caso práctico 1: “Cierre de ficheros” 7

/ 5. Serialización 8

/ 6. SerialVersiónUID 9

/ 7. Escritura de objetos 10

/ 8. Lectura de objetos 11

/ 9. Documentos XML 12

9.1. Formato XML 12

9.2. Parser XML 13

/ 10. Documentos JSON 14

/ 11. Caso práctico 2: “Texto o binario” 15

© MEDAC ISBN: 978-84-19885-69-2
Reservados todos los derechos. Queda rigurosamente prohibida, sin la autorización escrita de los titulares del copyright, bajo las sanciones establecidas en las leyes, la reproducción, transmisión y distribución total o parcial de esta obra por cualquier medio o procedimiento, incluidos la reprografía y el tratamiento informático.

# ÍNDICE

/ 12. Documentos DTD Y XSD 16

/ 13. Biblioteca DOM 17

13.1. Uso de la biblioteca DOM 18

/ 14. Resumen y resolución del caso práctico de la unidad 18

/ 15. Bibliografía 20

# OBJETIVOS

*Leer información de ficheros binarios.*

*Almacenar información en ficheros binarios.*

*Leer objetos de ficheros.*

*Serializar archivos.*

*Almacenar objetos en ficheros.*

*Conocer distintas estructuras jerárquicas anidadas.*

## / 1. Introducción y contextualización práctica

En esta unidad se va a profundizar sobre el concepto de fichero. Al igual que con los ficheros de texto, vamos a estudiar cómo podemos escribir y leer información de ficheros binarios.

También vamos a aprender a almacenar y obtener objetos directamente de ficheros, es decir, lo que se conoce como la serialización de objetos.

Por último, nos detendremos en conocer distintos tipos de ficheros que comparten una estructura jerárquica anidada.

A continuación, vamos a plantear un caso práctico a través del cual podremos aproximarnos de forma práctica a la teoría de este tema.

Escucha el siguiente audio donde planteamos la contextualización práctica de este tema. Encontrarás su resolución en el apartado 'Resumen y resolución del caso práctico'.




*Fig.1. Serialización de Java.*

*Audio intro. "Utilidad de ficheros"*

*https://bit.ly/3jRcLKN*

# / 2. Lectura de ficheros binarios

Para leer los datos de un fichero binario **necesitamos las siguientes clases**:

- **FileInputStream**: Case en Java que permite leer *bytes* de un archivo en el sistema de archivos. Es una subclase de InputStream y se utiliza para leer datos binarios, como imágenes, audio, vídeo, etc. Esta clase es útil cuando necesitamos leer datos byte por byte, lo que es común en la lectura de archivos binarios.

- **DataInputStream**: Clase de utilidad en Java que permite leer datos primitivos de una secuencia de entrada. Esta clase es una subclase de FileInputStream y está diseñada para leer datos en formato primitivo (por ejemplo, int, float, double, etc.) desde una secuencia de entrada, como un archivo binario.

Los **pasos** a seguir para **leer datos de un fichero binario** son los siguientes:

1. **Crear una instancia de FileInputStream**: Hay que crear un objeto FileInputStream para abrir el archivo binario que se desea leer:

```
FileInputStream fileInputStream = new FileInputStream("archivo.bin");
```

2. **Crear una instancia de DataInputStream**: Se debe envolver el FileInputStream en un objeto DataInputStream para permitir la lectura de tipos de datos primitivos:

```
DataInputStream dataInputStream = new DataInputStream(fileInputStream);
```

3. **Leer datos del archivo**: Hay que utilizar los métodos proporcionados por DataInputStream para leer los datos del archivo binario. Es posible usar métodos como readInt(), readFloat(), readDouble(), etc., dependiendo del tipo de datos que se espere leer:

```
int intValue = dataInputStream.readInt();
float floatValue = dataInputStream.readFloat();
```

4. **Cerrar los flujos de entrada**: Después de leer el archivo, hay que asegurarse de cerrar los flujos de entrada para liberar recursos:

```
dataInputStream.close();
fileInputStream.close();
```

La siguiente imagen muestra un ejemplo que combina todos estos pasos:

```java
import java.io.DataInputStream;
import java.io.FileInputStream;
import java.io.IOException;

public class BinaryFileReader {
    public static void main(String[] args) {
        String filePath = "archivo.bin";

        try {
            FileInputStream fileInputStream = new FileInputStream(filePath);
            DataInputStream dataInputStream = new DataInputStream(fileInputStream);

            // Leer datos del archivo binario
            int intValue = dataInputStream.readInt();
            float floatValue = dataInputStream.readFloat();

            // Imprimir los valores leídos
            System.out.println("Valor entero leído: " + intValue);
            System.out.println("Valor flotante leído: " + floatValue);

            // Cerrar flujos después de su uso
            dataInputStream.close();
            fileInputStream.close();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

*Fig.2. Lectura de un archivo binario.*

**Recuerda...**

La detección del fin del archivo se realiza típicamente mediante la comprobación de si el método read() devuelve -1, lo cual indica que se ha alcanzado el final del archivo.

# / 3. Escritura de ficheros binarios

Los ficheros de binarios son aquellos cuya **codificación es en bytes**, esto quiere decir que **no podremos editarlos con cualquier editor de texto**, como el bloc de notas. Aunque es verdad que cuando los tengamos preparados podremos abrirlos con editores que contienen soporte para la codificación binaria.

Si bien es cierto que, de manera previa, la lectura de ficheros binarios la hemos realizado con las clases FileInputStream y DataInputStream y de manera análoga podría realizarse con las clases FileOutputStream y DataOutputStream, en este caso vamos a emplear la **clase BufferOutputStream**.

**Para escribir en un fichero binario, necesitamos las siguientes clases**:

- **FileOutputStream**: Con ella podremos generar un flujo de escritura carácter a carácter a un fichero binario.
- **BufferedOutputStream**: Con esta podremos generar un flujo de escritura línea a línea a un fichero binario.

En este tipo de ficheros, cuando escribamos una línea, no se agregará el salto de línea al final, sino que tendremos que agregarlo nosotros manualmente. La siguiente imagen muestra los **pasos a seguir para escribir en un fichero binario**:

1. **Crear un objeto de tipo FileOutputStream a partir de la ruta del fichero**: Con esto crearemos el flujo de escritura hacia el fichero, pero solo podremos escribir carácter a carácter.
2. **Crear un objeto de tipo BufferedOutputStream a partir del objeto FileOutputStream**: Para poder escribir cadenas de texto completas en una operación y mejorar la eficiencia de escritura al utilizar un búfer.

3. **Escribir las cadenas que necesitemos a partir del método write de la clase BufferedOutputStream**: Tenemos que indicar el tipo de codificación que queremos usar, en este caso será la codificación UTF8, mediante la clase StandardCharsets. Como estos ficheros están codificados en bytes, tendremos que obtener los bytes de la cadena mediante el método getBytes de la clase String.

4. **Cerrar el fichero**: Para ello utilizamos el método close de las clases FileOutputStream y BufferedOutputStream.

```java
import java.nio.charset.StandardCharsets;
import java.io.*;

public class CharsetExample {
    public static void main(String[] args) {
        String filePath = "archivo.txt";

        try {
            // Escribir datos en un archivo utilizando UTF-8
            FileOutputStream fileOutputStream = new FileOutputStream(filePath);
            OutputStreamWriter outputStreamWriter = new OutputStreamWriter(fileOutputStream, StandardCharsets.UTF_8);
            BufferedWriter bufferedWriter = new BufferedWriter(outputStreamWriter);
            bufferedWriter.write("¡Hola mundo!");
            bufferedWriter.close();

        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

*Fig.3. Escritura de un archivo binario.*

*Vídeo 1. "Escritura en un fichero binario"*

https://bit.ly/2Z6tVMg

# / 4. Caso práctico 1: "Cierre de ficheros"

**Planteamiento**: El cierre de ficheros en Java es crucial para liberar recursos del sistema y evitar fugas de memoria, garantizando que los datos se guarden correctamente y evitando la corrupción de archivos. Además, asegura que los descriptores de ficheros se liberen, permitiendo que otros procesos accedan a ellos y evitando bloqueos que podrían afectar la concurrencia y la estabilidad del sistema.

**Nudo**: Se ha realizado un programa que tiene que pedir 10 números y almacenarlos en un fichero de texto. Una vez realizadas las distintas operaciones con ficheros, se decide cerrarlo. Ya hemos visto que el manejo de ficheros lo hacemos dentro de un bloque *try-catch* y surge la duda de si cerrar el fichero en el catch o en el *finally*. Si se puede cerrar el fichero tanto en el try como en el catch, ¿es innecesario el bloque finally para cerrarlo?

**Desenlace**: El cierre de ficheros después de trabajar con él es una de las partes más importantes, ya que, si no se cerrase, podrían producirse errores, como que no se guarde lo que se ha desarrollado o almacenado.

Cuando se lanza una excepción, todo el código que hay por debajo no se ejecutará, yéndose al bloque catch a ejecutar el código de tratamiento de errores. Bien, puede ocurrir que, si ha saltado una excepción y estamos en el bloque catch, suceda de nuevo otra excepción y la orden de cerrar el fichero que había preparada no se ejecute, quedándose este abierto con todo lo que ello pudiera conllevar. Por lo cual, es obligatoria la implementación del cierre del fichero dentro del bloque finally, para asegurar su cierre.

```java
} catch (IOException e) {
    System.out.println("Error leyendo el fichero: " + e.toString());
} finally {
    // Este bloque se ejecutará siempre, haya excepción o no.
    // Aquí es dónde se debe cerrar el archivo.
}
```

*Fig.4. Cierre de ficheros.*

# / 5. Serialización

**Mediante la serialización** vamos a poder escribir objetos en los ficheros, esto quiere decir que, en lugar de escribir dato por dato todas las propiedades de un objeto, **podremos escribir el objeto directamente**, ahorrándonos muchos accesos a disco innecesarios.

Para que un objeto pueda ser serializado tiene que implementar obligatoriamente la **interfaz Serializable**, en caso contrario, cuando nos dispongamos a escribirlo en el fichero provocará un error en nuestro programa.

Los ficheros en los que vamos a escribir los objetos son ficheros binarios, por lo que vamos a poder aprovechar lo estudiado en los puntos anteriores.

Si bien previamente, cuando habíamos creado un fichero binario, lo podíamos abrir sin problema con un editor de texto, en este caso no pasará lo mismo, ya que, debido a la configuración de la interfaz Serializable, **el fichero, al abrirlo, será indescifrable** y solo veremos una serie de números hexadecimales que no tendrán ningún significado para nosotros, pero que están codificando toda la información de los objetos.

Cuando se serializan los objetos, todos sus atributos serán escritos, pero podemos **indicar que un atributo no se serialice**, mediante el **modificador *transient***. Cuando declaramos una variable como *transient*, no guarda su valor al ser serializado un objeto de esa clase, teniendo que volver a asignarlo manualmente al recuperarlo.

Para poder utilizar la interfaz Serializable tendremos que hacer el siguiente *import*:

```
import java.io.Serializable;
```

**En el caso de que queramos serializar una clase que tenga como atributo un objeto, ese objeto tendrá que ser serializable obligatoriamente**.

La siguiente imagen muestra un ejemplo de una clase que se podrá serializar:

```java
import java.io.Serializable;

/**
 * Esta clase representa una persona
 * @author Francisco Jiménez Moral
**/

public class Persona implements Serializable {

    private String nombre;
    private String dni;
    private transient int edad;

    // Constructor
    public Persona(String nombre, String dni, int edad) {
        this.nombre = nombre;
        this.dni = dni;
        this.edad = edad;
    }
```

*Fig.5. Clase Persona serializable.*

### ¡! Sabías que...

Si una clase serializable tiene una subclase, la subclase también es serializable automáticamente, a menos que se indique explícitamente lo contrario.

# / 6. SerialVersiónUID

Cuando trabajamos con **serialización de objetos**, tenemos que tener mucho **cuidado con las versiones de nuestros programas**.

Ya sabemos de unidades anteriores que cuando compilamos nuestro proyecto se compilarán todos los ficheros Java que tengamos y se crearán ficheros .class, que son los que tienen el código intermedio que es capaz de ejecutar la máquina virtual. Cada vez que compilamos, esos ficheros de código intermedio se van actualizando. Hasta aquí todo bien, pero ¿qué ocurre con la serialización?

Cuando de un fichero binario recuperamos un objeto que fue serializado previamente, **se comprobará que la clase de la instancia del objeto recuperado sea exactamente igual que la que había cuando se instanció el objeto antes de serializarlo**.

Puede ocurrir que el proyecto haya cambiado y algunos de esos cambios se hayan efectuado en la clase del objeto serializado (podemos haberle cambiado algún atributo, haberlo eliminado o haberle agregado algún atributo extra porque se necesitase cubrir alguna necesidad).

Estas **comprobaciones se hacen mediante un número de versión**, que, si nosotros no lo indicamos, lo hará automáticamente el compilador. No obstante, cuando haya cambiado la clase, el compilador no cambiará la versión correctamente, de modo que al intentar recuperar un objeto cuya clase era diferente cuando se serializó, tendremos un error.

Para evitar este problema, **se aconseja que la clase del objeto que se serializará tenga un atributo privado**, de esta forma:

```
private static final long serialVersionUID = 1L;
```

Así, **el número que le asignemos será diferente en cada versión de la clase** y la clase Persona que serializamos previamente quedará de la siguiente manera (imagen):

```java
import java.io.Serializable;

/**
 * Esta clase representa una persona
 * @author Francisco Jiménez Moral
**/

public class Persona implements Serializable {

    private static final long serialVersionUID = 1L;
    private String nombre;
    private String dni;
    private transient int edad;

    // Constructor
    public Persona(String nombre, String dni, int edad) {
        this.nombre = nombre;
        this.dni = dni;
        this.edad = edad;
    }
```

*Fig.6. SerialVersionUID de la clase Persona.*

### ¡! Sabías que...

Los cambios en la estructura de la clase (como la adición o eliminación de campos) pueden romper la compatibilidad entre versiones de una clase serializable a menos que se maneje cuidadosamente usando serialVersionUID y métodos de serialización personalizados.

# / 7. Escritura de objetos

Para **serializar objetos en un fichero binario** necesitamos las siguientes **clases**:

- **FileOutputStream**: Con ella podremos generar un flujo de escritura carácter a carácter a un fichero binario.
- **ObjectOutputStream**: Con esta podremos generar un flujo de escritura de objetos a un fichero binario.

En este ejemplo, se asume que la clase del objeto que se va a escribir en el archivo objeto.dat implementa la interfaz Serializable para que pueda ser serializado correctamente.

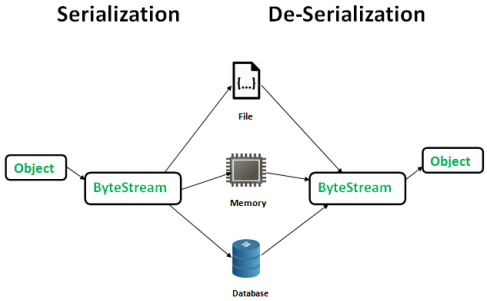



*Fig.7. Apertura de fichero serializado.*

Los **pasos a seguir para escribir en un fichero binario** son los siguientes:

1. **Crear un objeto de tipo FileOutputStream a partir de la ruta del fichero**: Con esto crearemos el flujo de escritura hacia el fichero, pero solo podremos escribir carácter a carácter.
2. **Crear un objeto de tipo ObjectOutputStream a partir del objeto FileOutputStream**: Para poder serializar objetos en el fichero.
3. **Escribir todos los objetos de una clase que implemente la interfaz Serializable mediante el método writeObject de la clase ObjectOutputStream**: Los objetos se irán escribiendo uno detrás de otro en nuestro fichero binario.
4. **Cerrar el fichero**: Para ello utilizamos el método close de las clases FileOutputStream y ObjectOutputStream. Es importante cerrar primero el objeto de ObjectOutputStream y después el FileOutputStream.

Al igual que con los demás tipos de ficheros, podremos indicar que se empiece a escribir la información en el final del fichero, no sobrescribiendo la información que ya hubiese, como se muestra a continuación en la imagen:

```
import java.io.FileInputStream;
import java.io.ObjectInputStream;
import java.io.IOException;

public class ObjectReader {
    public static void main(String[] args) {
        String filePath = "objeto.dat";

        try (FileInputStream fileInputStream = new FileInputStream(filePath);
             ObjectInputStream objectInputStream = new ObjectInputStream(fileInputStream)) {

            // Leer el objeto desde el archivo
            Object objetoLeido = objectInputStream.readObject();

            // Realizar un casting al tipo de objeto adecuado
            MiObjeto miObjeto = (MiObjeto) objetoLeido;

            // Ahora puedes trabajar con el objeto leído
            System.out.println("Objeto leído: " + miObjeto);

        } catch (IOException | ClassNotFoundException e) {
            System.out.println(e.toString());
        } finally {
            objectInputStream.close();
            fileInputStream.close();
            catch(IOException){
                System.out.println(e.toString());
            }
        }
    }
}
```

*Fig.8. Escritura de objetos.*

En el siguiente vídeo podrás aprender más sobre el apartado anterior.



*Vídeo 2. "Serialización de objetos"*
*https://bit.ly/3bsNT94*



# / 8. Lectura de objetos

Para **leer los objetos serializados en un fichero binario** necesitamos las siguientes **clases**:

- **FileInputStream**: Con ella podremos generar un flujo de lectura carácter a carácter a un fichero binario.
- **ObjectInputStream**: Con esta podremos generar un flujo de lectura de objetos a un fichero binario.

Los **pasos a seguir para leer datos de un fichero binario** son los siguientes:

1. **Creamos un objeto de tipo FileInputStream**:

```
FileInputStream fileInputStream = new FileInputStream(filePath);
```

2. **Creamos un objeto de tipo ObjectInputStream a partir del objeto de tipo FileInputStream**: Con esto podremos leer objetos directamente:

```
ObjectInputStream objectInputStream = new ObjectInputStream(fileInputStream))
```

3. **Leemos los objetos que necesitemos**: Para leer un objeto utilizaremos el método readObject de la clase ObjectInputStream, el cual nos devolverá un dato de tipo Object, por lo que debemos hacer un *casting* al tipo de objeto que nosotros necesitemos:

```
Object objetoLeido = objectInputStream.readObject();
```

4. **Cerramos el fichero**: Para ello utilizamos el método close de las clases FileInputStream y ObjectInputStream. Es importante cerrar primero el objeto de ObjectInputStream y después el de FileInputStream:

```
objectInputStream.close();
fileInputStream.close();
```

Al igual que pasaba con los demás ficheros, **en caso de que el fichero que queramos leer no exista, se producirá un error**.

```
import java.io.FileOutputStream;
import java.io.ObjectOutputStream;
import java.io.IOException;

public class ObjectWriter {
    public static void main(String[] args) {
        String filePath = "objeto.dat";

        // Crear un objeto que se va a escribir en el archivo
        MiObjeto objetoAEscribir = new MiObjeto("Datos de ejemplo");

        try (FileOutputStream fileOutputStream = new FileOutputStream(filePath);
             ObjectOutputStream objectOutputStream = new ObjectOutputStream(fileOutputStream)) {

            // Escribir el objeto en el archivo
            objectOutputStream.writeObject(objetoAEscribir);
            System.out.println("Objeto escrito en el archivo correctamente.");

        } catch (IOException e) {
            System.out.println(e.toString());
        } finally {
            objectOutputStream.close();
            fileOutputStream.close();
            catch(IOException){
                System.out.println(e.toString());
            }
        }
    }
}
```

*Fig.9. Lectura de objetos.*

# / 9. Documentos XML

El **XML** (Extensible Markup Language) es un **metalenguaje** que nos va a permitir **definir marcas para poder**, de esta forma, **almacenar datos de forma comprensible**. Se trata de un estándar utilizado en muchos campos, desde bases de datos, hasta documentos utilizados por instituciones oficiales.

Los documentos XML tienen las siguientes **ventajas**:

- **Se pueden modificar**, agregando nuevas etiquetas para nueva información, de una forma muy sencilla.
- **Hay verificadores de lenguaje XML ya disponibles** para no tener que crearlos de cero.
- **Su estructura es muy sencilla**, por lo que empezar a utilizarlos es muy fácil.

## 9.1. Formato XML

Los **documentos XML** se componen de las siguientes **partes**:

- **Prólogo**: Describe la versión de XML que estamos utilizando o la codificación. Su uso no es obligatorio.
- **Cuerpo**: Es donde se va a definir el elemento raíz del XML que contendrá todos los nodos con la información.
- **Elementos**: Nodos e información del XML.
- **Atributos**: Son propiedades de los nodos. Su uso no es obligatorio.
- **Elementos predefinidos**: Códigos para representar unos ciertos caracteres especiales.
- **Sección CDATA**: Esta sección permitirá definir una serie de caracteres para poder utilizarlos en el XML.
- **Comentarios**: Típicos comentarios que sirven para aclarar qué se está haciendo.

A continuación se muestra una figura del XML con la que vamos a realizar la lectura del fichero:

```xml
<?xml version="1.0" encoding="UTF-8" ?>
    <asignatura id ="21">
        <nombre>Programacion</nombre>
        <alumnos>29</alumnos>
        <aula>5</aula>
        <ciclo>DAM</ciclo>
    </asignatura>
```

*Fig.10. Formato XML de asignaturas.*

Para realizar la lectura del documento XML, se va a emplear **JAXB** (Java Architecture for XML Binding), que **permite convertir un documento XML en objetos Java y viceversa**. Esto implica generar clases Java a partir de un esquema XML o un documento XML existente (proceso conocido como generación de clases JAXB o *binding*).

## 9.2. Parser XML

Partimos de que el elemento del XML tiene su propia clase Asignatura y los elementos del XML nombre, alumnos, aula, ciclo e id se corresponden con sus atributos.

```java
public class Asignatura {
    private Integer identificación;
    private String nombreAsignatura;
    private String alumnos;
    private String aulaReferencia;
    private String ciclo;

    public Asignatura(Integer identificacion, String nombreAsignatura, String alumnos, String aulaReferencia, String ciclo){
        this.identificación = identificación;
        this.nombreAsignatura = nombreAsignatura;
        this.alumnos = alumnos;
        this.aulaReferencia = aulaReferencia;
        this.ciclo = ciclo;
    }
}
```

*Fig.11. Clase Asignatura.*

El siguiente paso es **añadir indicaciones para la conversión entre XML y Java para cada uno de los atributos**. Para ello hay que seguir los siguientes pasos:

1. **Importar la API JAXB**:

```
Import javax.xml.bind.annotation.*;
```

2. **Escribir justo encima de la definición de la clase Asignatura las siguientes líneas** que se muestran en la imagen:

```java
@XmlRootElement(name = asignatura)//el elemento raíz se llamará asignatura
@XmlType(propOrder = {nombreAsignatura, alumnos, aulaReferencia, ciclo})//Atributos de la clase Java
@XmlAccessorType(XmlAccessType.FIELD)

public class Asignatura {
```

*Fig.12. Anotaciones binding.*

Hay que escribir anotaciones encima de las declaraciones de los atributos de la clase para asociarlos al documento XML.

El primero será la **identificación**:

```
@XmlAttribute(name = "id", required = true)
private Integer identificación;
```

El **siguiente atributo** será:

```
@XmlElement(name = "nombre")
private String nombreAsignatura;
```

De la misma manera, **aula referencia**:

```
@XmlElement(name = "aula")
private String aulaReferencia;
```

**El resto de atributos no son necesarios, porque se llaman de la misma manera que el XML**.

3. A continuación **creamos el contexto de la clase JAXBContext al que le pasamos la clase que vamos a 'parsear' y el objeto Marshaller** (para escribir XML) **o Unmarshaller** (para leer XML), como se muestra en la imagen:

```
JAXBContext jaxbContext = JAXBContext.newInstance(Asignatura.class);
Unmarshaller unmarshaller = jaxbContext.createUnmarshaller();
```

*Fig.13. Creación de contexto.*

4. El siguiente paso es **construir el objeto Asignatura** con la siguiente sentencia (imagen)

```
Asignatura a = (Asignatura) unmarshaller.unmarshall(new File("asignaturas.json"));
```

*Fig.14. Lectura del objeto.*

5. Por último **imprimimos el objeto para comprobarlo**, para lo cual es necesario tener implementado de manera previa el método toString();

```
System.out.println(a);
```

*Fig.15. Impresión del objeto.*

# / 10. Documentos JSON

JSON (JavaScript Object Notation) es un **formato de intercambio de datos ligero que se puede interpretar como una representación de datos similar a un objeto o estructura de datos en JavaScript**. Sin embargo, Java no tiene un tipo de datos incorporado para representar directamente JSON, por lo que generalmente se utilizan **bibliotecas externas** como Gson, Jackson o JSON.simple.

**Para el ejemplo que vamos a realizar, se va a añadir la biblioteca JSON.simple**. Se trata de una solución simple y ligera sin depender de bibliotecas externas más grandes como Gson o Jackson.

El primer paso será **añadir la biblioteca json-simple-1.1.1.jar**, que puede descargarse en el siguiente enlace, y añadirla al proyecto. Para ello, iremos a la carpeta del proyecto en NetBeans, haremos clic derecho en Libraries y seleccionaremos Add JAR/Folder obteniendo el siguiente resultado (imagen):

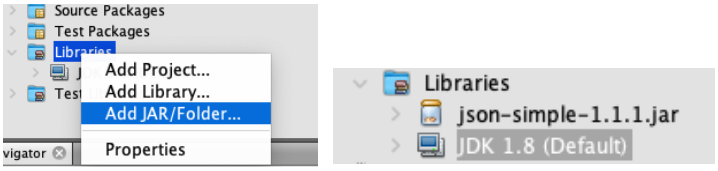



*Fig.16. Añadir librería json-simple-1.1.1.jar.*

Una vez **añadida la biblioteca**, vamos a realizar **dos operaciones**:

1. La **lectura de un JSON desde una cadena de texto** tal cual aparece en el código:

```
import org.json.simple.JSONObject;
import org.json.simple.parser.JSONParser;

public class Main {
    public static void main(String[] args) {
        JSONParser parser = new JSONParser();
        String jsonString = "{\"nombre\": \"Juan\", \"edad\": 30}";

        try {
            JSONObject jsonObject = (JSONObject) parser.parse(jsonString);
            String nombre = (String) jsonObject.get("nombre");
            long edad = (long) jsonObject.get("edad");

            System.out.println("Nombre: " + nombre);
            System.out.println("Edad: " + edad);
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

*Fig.17. Lectura de un JSON.*

2. **Generar un JSON a partir de objetos Java** como se muestra en el siguiente código:

```
import org.json.simple.JSONObject;

public class Main {
    public static void main(String[] args) {
        JSONObject jsonObject = new JSONObject();
        jsonObject.put("nombre", "María");
        jsonObject.put("edad", 25);

        String jsonString = jsonObject.toJSONString();
        System.out.println(jsonString);
    }
}
```

*Fig.18. Creación de un JSON.*

# / 11. Caso práctico 2: "Texto o binario"

**Planteamiento**: La elección entre fichero de texto y fichero binario depende del contexto de uso. Los ficheros de texto son preferibles por su legibilidad y facilidad de edición, resultan ideales para configuraciones o datos legibles. Los ficheros binarios, por otro lado, son más eficientes en almacenamiento y velocidad de acceso, resultan adecuados para grandes volúmenes de datos o formatos específicos.

**Nudo**: Se está desarrollando una aplicación para la barra de un bar en la que se deben guardar los pedidos en un fichero, para que no se pierdan y poder recuperarlos cuando el cliente decida pagar la cuenta. ¿Qué piensas al respecto? ¿Cómo crees que sería mejor guardar la información, dato por dato o directamente el objeto mediante la serialización?

**Desenlace**: Al guardar información hay que pensar bien cómo queremos hacerlo, ya que eso afectará al diseño del programa.

Una de las cosas que nos tenemos que preguntar en este punto es cuánta información queremos almacenar. En el caso de que sea poca, unos cuantos números, alguna cadena, etc., no merece la pena serializarla; pero si contamos con mucha información, y además estructurada en clases, la mejor opción es hacer que esas clases puedan serializarse y almacenar en bloque, ya que luego la forma de recuperarlas es muchísimo más simple y rápida.

Por tanto, la respuesta al almacenamiento de la información para la barra de un bar sería guardar la información dato por dato en lugar de guardar un pedido mediante serialización.

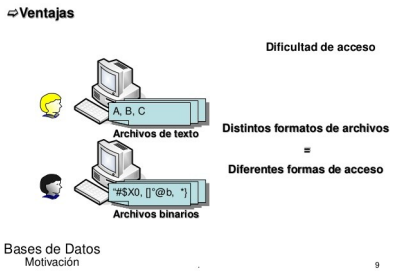



*Fig.19. Formato de archivo.*

# / 12. Documentos DTD Y XSD

Los **DTD** (Definición de Tipo de Documento) son documentos que **definen la descripción de los elementos de un fichero XML y su sintaxis**.

Su función consiste en **definir las estructuras de los elementos que se van a usar en el fichero XML** y que también validarán, consiguiendo así una consistencia en todos sus elementos, es decir, que todos los elementos del mismo tipo sean iguales.

Las definiciones de los DTD pueden hacerse en el mismo fichero XML que validarán, aunque lo más recomendable es hacerlo en un fichero aparte e indicarle posteriormente al fichero XML quién es su validador DTD.

Los **DTD van a describir**:

- **Los elementos**: Se indicará qué elementos están permitidos.
- **La estructura**: Se señalará el orden de los elementos en el documento XML.
- **El anidamiento de elementos**: Se indicará qué elementos van dentro de otros.

```
<!ELEMENT biblioteca (libro*)>
<!ELEMENT libro (ISBN, titulo, autor, editorial?)>
<!ELEMENT ISBN (#PCDATA) >
<!ELEMENT titulo (#PCDATA) >
<!ELEMENT autor (#PCDATA) >
<!ELEMENT editorial (#PCDATA)>
```

*Fig.20. Ejemplo de validador DTD.*

Los **documentos XSD** se utilizan **para definir la estructura y las restricciones de los documentos XML de una forma precisa**. El XSD es un lenguaje que está escrito en XML y proporciona una mayor potencia que los DTD.

Estos se desarrollaron como una **alternativa mejor a los DTD**, mucho más completa, que **intenta arreglar los puntos débiles** de estos.

La definición de los XSD se basa en lo que se conoce como ***namespaces* o espacios de nombre**. Cada uno de ellos va a contener una serie de elementos relacionados entre sí, siendo algo parecido a los paquetes en Java.

Una vez escrito un XSD, habrá que validarlo para comprobar que no existe ningún error con él. Para ello, a la hora de validar documentos XML contra un esquema XSD (XML Schema Definition) en Java, podemos utilizar las clases del paquete javax.xml.validation.

```
<?xml version="1.0" encoding="UTF-8"?>
<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema">
    <xsd:element name="Libro">
        <xsd:complexType>
            <xsd:sequence>
                <xsd:element name="Titulo" type="xsd:string"/>
                <xsd:element name="Autores" type="xsd:string" maxOccurs="10"/>
                <xsd:element name="Editorial" type="xsd:string"/>
            </xsd:sequence>
            <xsd:attribute name="precio" type="xsd:double"/>
        </xsd:complexType>
    </xsd:element>
</xsd:schema>
```

*Fig.21. Ejemplo de fichero XSD.*

# / 13. Biblioteca DOM

Es una **biblioteca para tratamiento de ficheros XML optimizado para el lenguaje de programación Java**. Ofrece un tratamiento eficiente y transparente de ficheros XML, ya que estos pueden llegar a formar estructuras complejas, siendo difíciles de tratar si no se impone cierto orden.

Esta biblioteca es similar a la DOM que se utiliza en JavaScript, solo que está **especialmente creada y diseñada para el lenguaje de programación Java**.

Es una **biblioteca interna**, lo que quiere decir que ya está integrada en la propia JDK de Java, así que podremos utilizarla sin necesidad de agregar nada a nuestros proyectos.

La **biblioteca estándar** de Java para trabajar con DOM es el **paquete javax.xml.parsers**, que proporciona clases y métodos para analizar, crear, manipular y transformar documentos XML empleando el estándar DOM.

Para poder utilizarla necesitaremos el siguiente import:

```
import org.w3c.dom.*;
```

Si bien no utilizaremos el asterisco para importar todas las clases. **Dentro de esta biblioteca tendremos las siguientes clases**:

- **Document**: Representa un documento XML.
- **Node**: Representa un nodo XML.
- **NodeList**: Representa una lista de nodos XML.
- **Element**: Representa un elemento del XML. Puede ser un nodo.
- **DOMImplementation**: Representa la configuración de XML.
- **Text**: Representa el valor de un nodo.
- **Comment**: Representa un comentario.

## 13.1. Uso de la biblioteca DOM

A continuación, se muestra una figura con un ejemplo de uso de la biblioteca DOM:

```
//------------Obtener un Analizador de XML (Parser)------------

import javax.xml.parsers.DocumentBuilderFactory;
import javax.xml.parsers.DocumentBuilder;
import org.w3c.dom.Document;

DocumentBuilderFactory factory = DocumentBuilderFactory.newInstance();
DocumentBuilder builder = factory.newDocumentBuilder();


//------------Analizar un Documento XML------------
Document document = builder.parse(new File("documento.xml"));


//------------Trabajar con el Documento DOM------------
// Obtener el elemento raíz
Element rootElement = document.getDocumentElement();


// Obtener elementos hijos
NodeList nodeList = rootElement.getChildNodes();

// Recorrer los elementos hijos
for (int i = 0; i < nodeList.getLength(); i++) {
    Node node = nodeList.item(i);
    if (node.getNodeType() == Node.ELEMENT_NODE) {
        Element element = (Element) node;
        // Trabajar con los elementos...
    }
}

//------------Modificar el Documento DOM------------

// Crear un nuevo elemento
Element newElement = document.createElement("nuevoElemento");
newElement.setAttribute("atributo", "valor");

// Añadir el nuevo elemento como hijo del elemento raíz
rootElement.appendChild(newElement);


//------------Escribir el Documento DOM Modificado a un Archivo XML------------
TransformerFactory transformerFactory = TransformerFactory.newInstance();
Transformer transformer = transformerFactory.newTransformer();
DOMSource source = new DOMSource(document);
StreamResult result = new StreamResult(new File("documento_modificado.xml"));
transformer.transform(source, result);
```

*Fig.22. Uso de la biblioteca DOM.*



*Audio 1. "Biblioteca DOM"*
*https://on.soundcloud.com/HYfU9sEGai68KDMj6*



# / 14. Resumen y resolución del caso práctico de la unidad

A lo largo de este tema se ha visto que podemos realizar **operaciones de escritura y lectura** no solo con los ficheros de texto, sino también **con los binarios**; aunque estos tengan una codificación diferente, podemos tanto escribir como leer información de ellos.

Se ha comprobado que en el **bloque try-catch** es necesario agregar un nuevo bloque, el bloque finally, que se ejecutará cuando termine el bloque try o catch indistintamente, y que nos servirá para cerrar los ficheros adecuadamente.

También se ha aprendido a escribir y leer objetos directamente de ficheros, mediante la **serialización**, y que para que esto funcione la clase de la que queremos serializar los objetos tendrá que implementar la interfaz Serializable.

Por último, se han introducido distintos tipos de ficheros, como son XML, JSON, DTD o XSD que, aunque tienen en común ser estructuras jerárquicas anidadas, se opera de manera distinta en cada una de ellas.

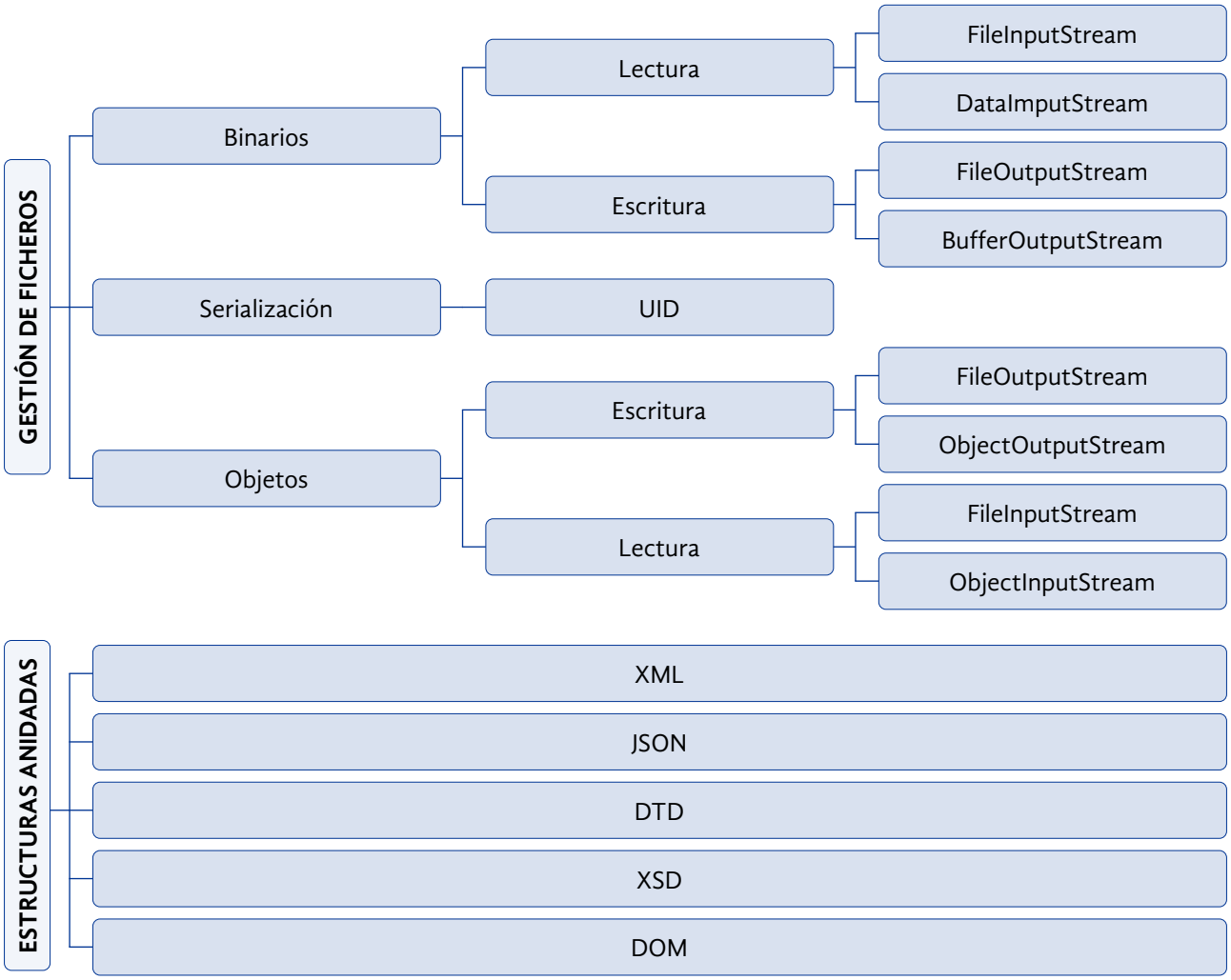



*Fig.23. Esquema resumen del tema.*

## Resolución del caso práctico de la unidad

Es posible que cuando empecemos a estudiar la utilización de ficheros en programación nos hagamos la misma pregunta: si ya existen las bases de datos, ¿para qué necesitamos almacenar la información en un fichero?

En cierto modo tienen razón. Posiblemente, el almacenamiento en bases de datos sea más eficiente y robusto que en ficheros, pero tenemos que ponernos en situación.

Imaginemos que necesitamos guardar cierta información en nuestro programa, pero no es una cantidad de información grande, sino que se trata de unos datos que necesitaremos en la siguiente ejecución del programa, ¿vale la pena montar una base de datos, con todo lo que ello conllevará, para almacenar unos pocos números?

La respuesta es no, así que para almacenar una cierta información no muy extensa, siempre nos serán de gran utilidad los ficheros.

# / 15. Bibliografía

Bloch, J. (2018). *Effective Java* (3.ª ed.). Addison-Wesley.

Caro, C. M., Ramos, A. N. y Barceló, A. V. (2002). *Introducción a la programación con orientación a objetos*. Prentice Hall.

Stack Overflow en español (s. f.). *¿Cómo leer correctamente los datos de un Json en Java?* Recuperado de: https://es.stackoverflow.com/questions/443785/c%C3%B3mo-leer-correctamente-los-datos-de-un-json-en-java

UPV/EHU (s. f.). *Flujos de datos*. Recuperado de: http://www.sc.ehu.es/sbweb/fisica/cursoJava/fundamentos/archivos/flujos.htm

Knuth, D. E. (1997). *The Art of Computer Programming* (Vol. 1: *Fundamental Algorithms*) (3.ª ed.). Addison-Wesley.

Manuel, P. G. J. (2022). *Entornos de desarrollo*. Ediciones Paraninfo, S.A.

Ojeda, F. C. (2001). *Introducción a la programación*. Anaya Multimedia.

Parzibyte (2019, 11 de febrero). *Añadir librerías o archivos JAR a NetBeans*. Recuperado de Parzibyte's blog: https://parzibyte.me/blog/2019/02/15/anadir-librerias-archivos-jar-netbeans/

Schildt, H. (2018). *Java: The complete reference* (11.ª ed.). McGraw-Hill Education.